PEP 723 – Inline script metadata

Important

This PEP is a historical document. The up-to-date, canonical spec, Inline script metadata, is maintained on the PyPA specs page.

See the PyPA specification update process for how to propose changes.

Abstract

This PEP specifies a metadata format that can be embedded in single-file Python scripts to assist launchers, IDEs and other external tools which may need to interact with such scripts.

Motivation

Python is routinely used as a scripting language, with Python scripts as a (better) alternative to shell scripts, batch files, etc. When Python code is structured as a script, it is usually stored as a single file and does not expect the availability of any other local code that may be used for imports. As such, it is possible to share with others over arbitrary text-based means such as email, a URL to the script, or even a chat window. Code that is structured like this may live as a single file forever, never becoming a full-fledged project with its own directory and pyproject.toml file.

An issue that users encounter with this approach is that there is no standard mechanism to define metadata for tools whose job it is to execute such scripts. For example, a tool that runs a script may need to know which dependencies are required or the supported version(s) of Python.

There is currently no standard tool that addresses this issue, and this PEP does not attempt to define one. However, any tool that does address this issue will need to know what the runtime requirements of scripts are. By defining a standard format for storing such metadata, existing tools, as well as any future tools, will be able to obtain that information without requiring users to include tool-specific metadata in their scripts.

Rationale

This PEP defines a mechanism for embedding metadata within the script itself, and not in an external file.

The metadata format is designed to be similar to the layout of data in the pyproject.toml file of a Python project directory, to provide a familiar experience for users who have experience

writing Python projects. By using a similar format, we avoid unnecessary inconsistency between packaging tools, a common frustration expressed by users in the recent packaging survey.

The following are some of the use cases that this PEP wishes to support:

- A user facing CLI that is capable of executing scripts. If we take Hatch as an example, the interface would be simply hatch run /path/to/script.py [args] and Hatch will manage the environment for that script. Such tools could be used as shebang lines on non-Windows systems e.g. #!/usr/bin/env hatch run.
- A script that desires to transition to a directory-type project. A user may be rapidly prototyping locally or in a remote REPL environment and then decide to transition to a more formal project layout if their idea works out. Being able to define dependencies in the script would be very useful to have fully reproducible bug reports.
- Users that wish to avoid manual dependency management. For example, package managers that have commands to add/remove dependencies or dependency update automation in CI that triggers based on new versions or in response to CVEs [1].

Specification

This PEP defines a metadata comment block format loosely inspired [2] by reStructuredText Directives.

Any Python script may have top-level comment blocks that MUST start with the line # /// TYPE where TYPE determines how to process the content. That is: a single #, followed by a single space, followed by three forward slashes, followed by a single space, followed by the type of metadata. Block MUST end with the line # ///. That is: a single #, followed by a single space, followed by three forward slashes. The TYPE MUST only consist of ASCII letters, numbers and hyphens.

Every line between these two lines (# /// TYPE and # ///) MUST be a comment starting with #. If there are characters after the # then the first character MUST be a space. The embedded content is formed by taking away the first two characters of each line if the second character is a space, otherwise just the first character (which means the line consists of only a single #).

Precedence for an ending line # /// is given when the next line is not a valid embedded content line as described above. For example, the following is a single fully valid block:

```
# /// some-toml
# embedded-csharp = """
# /// <summary>
# /// text
# ///
# /// </summary>
# public class MyClass { }
```

```
# """
# ///
```

A starting line MUST NOT be placed between another starting line and its ending line. In such cases tools MAY produce an error. Unclosed blocks MUST be ignored.

When there are multiple comment blocks of the same TYPE defined, tools MUST produce an error.

Tools reading embedded metadata MAY respect the standard Python encoding declaration. If they choose not to do so, they MUST process the file as UTF-8.

This is the canonical regular expression that MAY be used to parse the metadata:

```
(?m)^# /// (?P<type>[a-zA-Z0-9-]+)$\s(?P<content>(^#(| .*)$\s)+)^# ///$
```

In circumstances where there is a discrepancy between the text specification and the regular expression, the text specification takes precedence.

Tools MUST NOT read from metadata blocks with types that have not been standardized by this PEP or future ones.

script type

The first type of metadata block is named script which contains script metadata (dependency data and tool configuration).

This document MAY include top-level fields dependencies and requires-python, and MAY optionally include a [tool] table.

The [tool] table MAY be used by any tool, script runner or otherwise, to configure behavior. It has the same semantics as the tool table in pyproject.toml.

The top-level fields are:

- dependencies: A list of strings that specifies the runtime dependencies of the script. Each entry MUST be a valid PEP 508 dependency.
- requires-python: A string that specifies the Python version(s) with which the script is compatible. The value of this field MUST be a valid version specifier.

Script runners MUST error if the specified dependencies cannot be provided. Script runners SHOULD error if no version of Python that satisfies the specified requires-python can be provided.

Example

The following is an example of a script with embedded metadata:

```
# /// script
# requires-python = ">=3.11"
# dependencies = [
#   "requests<3",
#   "rich",
# ]
# ///

import requests
from rich.pretty import pprint

resp = requests.get("https://peps.python.org/api/peps.json")
data = resp.json()
pprint([(k, v["title"]) for k, v in data.items()][:10])
```

Reference Implementation

The following is an example of how to read the metadata on Python 3.11 or higher.